15 動態規劃 解答

15-E01 rod cutting 成本

推導

r_0=0，r_1=2。r_2=max(5,2+2)=5。r_3=max(7,2+5,5+2)=7。
r_4=max(10,2+7,5+5,7+2)=10。r_5=max(12,2+10,5+7,7+5,10+2)=12。

結論與檢核

長 5 的最大收益 12。可用不切一段長 5，或由子問題得到其他等值切法；若要求固定段數，須把段數加入狀態。

15-E02 矩陣鏈轉移

推導

(A1A2)A3 先乘 A1A2 得 2・3・4=24，再乘 A3 得 2・4・2=16，共 40。A1(A2A3)：A2A3 得 3・4・2=24，再乘 A1 得 2・3・2=12，共 36。

結論與檢核

m[1,2]=24、m[2,3]=24，m[1,3]=min(24+2・4・2, 24+2・3・2)=36。最佳切點為 k=1，括號 A1(A2A3)。

15-E03 LCS 轉移與回溯

推導

以 columns 為 空、B、A、A，rows 為 空、A、B、A，完整長度表依序是 [0,0,0,0]、[0,0,1,1]、[0,1,1,1]、[0,1,2,2]。從 (3,3) 的共同 A 沿左上到 (2,2)，平手可往左到 (2,1) 的共同 B，反轉回溯字符得 BA。另一種平手路徑可得 AA。

結論與檢核

採平手先往上的規則，(3,3) 相等取 A 到 (2,2)；平手上到 (1,2)，相等取 A 到 (0,1)，反向得到 AA。若 (2,2) 平手向左，則再取 (2,1) 的 B，得到 BA。兩者長度均 2。每個移動都減少至少一個前綴長度，故必然終止。

15-E04 全域序列比對的狀態與回溯

推導

令 S[i,j] 為 X[1..i] 與 Y[1..j] 的最高分。基底 S[i,0]=-3i、S[0,j]=-3j；S[i,j]=max(S[i-1,j-1]+(相等 ? 3 : -2), S[i-1,j]-3, S[i,j-1]-3)。最佳對齊若以配對、X末配 gap、Y末配 gap 結尾，三者恰好覆蓋所有形式，刪去末欄後也必為較短前綴的最優比對，否則替換會提高總分。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

結論與檢核

S 的列由 X 的空前綴至 ACGTA、欄由 Y 的空前綴至 AGCAC，依序為 [0,-3,-6,-9,-12,-15]、[-3,3,0,-3,-6,-9]、[-6,0,1,3,0,-3]、[-9,-3,3,0,1,-2]、[-12,-6,0,1,-2,-1]、[-15,-9,-3,-2,4,1]。從 (5,5)=1 回溯依序為左至(5,4)、左上至(4,3)、左上至(3,2)、左上至(2,1)、上至(1,1)、左上至(0,0)，得到 X'=ACGTA-、Y'=A-GCAC。逐欄分數為 3-3+3-2+3-3=1，且 X'、Y' 各為原序列加一個 gap。只比較同長度前綴會漏掉插入或刪除造成的位移；這正是 LCS 轉移到一般 sequence alignment 時新增的得分模型，而狀態與重建仍使用同一套前綴 DP。

15-E05 無限供應背包的完整 DP

推導

令 f[c] 為總重量恰為 c 的最大價值，f[0]=0，其他未知先設 -inf；$f[c] = \max_{i:s_i \le c}(f[c - s_i] + v_i)$。最後答案取$\max_{0 \le c \le 13} f[c]$。若最優解最後加入物品 i，其前一步必為重量 c-s_i 的最優解；替換成更佳前綴會直接提高原解，故狀態轉移完備。

結論與檢核

f[0..13]=[0,-inf,-inf,5,7,-inf,10,12,14,15,17,19,21,22]。最大值 22 在容量 13，最後物品可取 B，剩 9 再取 A×3，故解為 A+A+A+B。按 value/weight，B=7/4=1.75 最高，greedy 取 B×3 得 21；DP 發現 22，證明比值 greedy 不正確。這個反例只否定一般比值 greedy，不能否定 unbounded knapsack 的所有 greedy 變體。本例容量 13 恰好也能裝滿，所以『至多 13』與『恰好 13』都得到 22；若規格允許留空，答案需對所有 f[c] 再取最大。時間 Theta(SC)、空間 O(C)，其中 S 是物品種類數、C 是容量。

15-E06 不均勻失敗機率的最佳搜尋樹

推導

空段 e[1,0]=0.50、e[2,1]=0.10、e[3,2]=0.10。w[1,1]=0.70，e[1,1]=0.50+0.10+0.70=1.30；w[2,2]=0.40，e[2,2]=0.10+0.10+0.40=0.60。w[1,2]=1。根 k1 的候選為 e[1,0]+e[2,2]+1=2.10；根 k2 的候選為 e[1,1]+e[3,2]+1=2.40。因此 root[1,2]=1。

結論與檢核

k1 為深度0的根，右子 k2 深度1，dummy d0 深度1，d1、d2 深度2。期望 =0.10・1+0.20・2+0.50・2+0.10・3+0.10・3=2.10。p2>p1 卻不應把 k2 放根，因高機率失敗區間 d0 也影響成本。若只計真實鍵比較，成本為2.10−(0.50+0.10+0.10)=1.40，最佳樹不變。

15 動態規劃 解答 （續）

15-E07 只有端點的最長簡單路徑狀態為何失敗

推導

u→z→v 長2，v→z→w 長2，兩者各自簡單；拼接 u→z→v→z→w 重複 z，故不是簡單路徑。真正簡單 u→w 路徑只有 u→z→w，長2。失敗原因是子解共用內部頂點，不是子解本身不夠長。

結論與檢核

可令 D[S,t] 為從固定起點 u 出發、恰使用頂點集合 S、終於 t 的最長路徑，D[{u},u]=0，其餘不可行為−∞。對邊 s→t 且 t∈S、s∈S\{t}，D[S,t]=max(D[S\{t},s]+w(s,t))。新增頂點 t 前不在較小集合，保證不重複；任一簡單路徑刪末點也落入此狀態，故完備。按 |S| 遞增填值，狀態 O(2^n n)，直接枚舉前驅為 O(2^n n^2)。恢復正確性付出了指數狀態，不能把它宣稱為多項式端點 DP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

原創驗算程式附錄: dp_revision.py

```
"""Exact traces for the Chapter 15 revision pages."""

from fractions import Fraction
from itertools import combinations, product


def rod_cut(prices):
    n = len(prices) - 1
    revenue = [0] * (n + 1)
    choice = [0] * (n + 1)
    for length in range(1, n + 1):
        best_value = float("-inf")
        best_first = 0
        for first in range(1, length + 1):
            candidate = prices[first] + revenue[length - first]
            if candidate > best_value:
                best_value = candidate
                best_first = first
        revenue[length] = best_value
        choice[length] = best_first
    return revenue, choice


def rebuild_rod(choice, n):
    pieces = []
    while n:
        pieces.append(choice[n])
        n -= choice[n]
    return pieces


def matrix_chain(p):
    n = len(p) - 1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 (續)

```
    inf = float("inf")
    m = [[0] * (n + 1) for _ in range(n + 1)]
    split = [[None] * (n + 1) for _ in range(n + 1)]
    for length in range(2, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            m[i][j] = inf
            for k in range(i, j):
                candidate = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j]
                if candidate < m[i][j]:
                    m[i][j] = candidate
                    split[i][j] = k
    return m, split


def parenthesize(split, i, j):
    if i == j:
        return f"A{i}"
    k = split[i][j]
    return f"({parenthesize(split, i, k)}{parenthesize(split, k + 1, j)})"


def lcs_trace(x, y):
    n, m = len(x), len(y)
    c = [[0] * (m + 1) for _ in range(n + 1)]
    direction = [[""] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if x[i - 1] == y[j - 1]:
                c[i][j] = c[i - 1][j - 1] + 1
                direction[i][j] = "diag"
            elif c[i - 1][j] >= c[i][j - 1]:
                c[i][j] = c[i - 1][j]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

```
                direction[i][j] = "up"
            else:
                c[i][j] = c[i][j - 1]
                direction[i][j] = "left"

    i, j = n, m
    result = []
    path = []
    while i and j:
        path.append((i, j, direction[i][j]))
        if direction[i][j] == "diag":
            result.append(x[i - 1])
            i -= 1
            j -= 1
        elif direction[i][j] == "up":
            i -= 1
        else:
            j -= 1
    return c, direction, "".join(reversed(result)), path


def optimal_bst(p, q):
    n = len(p)
    e = {}
    w = {}
    root = {}
    for i in range(1, n + 2):
        e[i, i - 1] = q[i - 1]
        w[i, i - 1] = q[i - 1]
    for length in range(1, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            w[i, j] = w[i, j - 1] + p[j - 1] + q[j]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

```
            e[i, j] = None
            for r in range(i, j + 1):
                candidate = e[i, r - 1] + e[r + 1, j] + w[i, j]
                if e[i, j] is None or candidate < e[i, j]:
                    e[i, j] = candidate
                    root[i, j] = r
    return e, w, root


def global_alignment(x, y, match=3, mismatch=-2, gap=-3):
    score = [[0] * (len(y) + 1) for _ in range(len(x) + 1)]
    move = [[""] * (len(y) + 1) for _ in range(len(x) + 1)]
    for i in range(1, len(x) + 1):
        score[i][0] = i * gap
        move[i][0] = "up"
    for j in range(1, len(y) + 1):
        score[0][j] = j * gap
        move[0][j] = "left"
    for i in range(1, len(x) + 1):
        for j in range(1, len(y) + 1):
            diagonal = score[i - 1][j - 1] + (
                match if x[i - 1] == y[j - 1] else mismatch
            )
            up = score[i - 1][j] + gap
            left = score[i][j - 1] + gap
            score[i][j] = max(diagonal, up, left)
            move[i][j] = "diag" if diagonal == score[i][j] else "up" if up == score[i][j]
else "left"

    i, j = len(x), len(y)
    a, b = [], []
    while i or j:
        direction = move[i][j]
```

15 動態規劃 解答 （續）

```
        if direction == "diag":
            a.append(x[i - 1])
            b.append(y[j - 1])
            i -= 1
            j -= 1
        elif direction == "up":
            a.append(x[i - 1])
            b.append("-")
            i -= 1
        else:
            a.append("-")
            b.append(y[j - 1])
            j -= 1
    return score, "".join(reversed(a)), "".join(reversed(b))


def unbounded_knapsack(items, capacity):
    best = [-1] * (capacity + 1)
    choose = [None] * (capacity + 1)
    best[0] = 0
    for cap in range(1, capacity + 1):
        for name, size, value in items:
            if size <= cap and best[cap - size] >= 0:
                candidate = best[cap - size] + value
                if candidate > best[cap]:
                    best[cap] = candidate
                    choose[cap] = name
    amount = max(range(capacity + 1), key=lambda cap: best[cap])
    chosen = []
    current = amount
    while current:
        name = choose[current]
        chosen.append(name)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 (續)

```
        current -= next(size for item_name, size, _ in items if item_name == name)
    return best, amount, chosen


def audit_small_dp():
    lcs_cases = 0
    words = ["".join(chars) for n in range(5) for chars in product("AB",repeat=n)]
    for x in words:
        subsequences={"".join(x[i] for i in ids) for k in range(len(x)+1)
                      for ids in combinations(range(len(x)),k)}
        for y in words:
            other={"".join(y[i] for i in ids) for k in range(len(y)+1)
                   for ids in combinations(range(len(y)),k)}
            table, _, result, _ = lcs_trace(x,y)
            assert result in subsequences & other
            assert len(result)==table[-1][-1]==max(map(len,subsequences & other))
            lcs_cases+=1

    def all_costs(p,i,j):
        if i==j:
            return [0]
        return [left+right+p[i-1]*p[k]*p[j]
                for k in range(i,j)
                for left in all_costs(p,i,k)
                for right in all_costs(p,k+1,j)]

    matrix_cases = 0
    for dims in product((2,3),repeat=6):
        table,_=matrix_chain(dims)
        assert table[1][5]==min(all_costs(dims,1,5))
        matrix_cases+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

```
    p=[Fraction(1,10),Fraction(2,10)]
    q=[Fraction(5,10),Fraction(1,10),Fraction(1,10)]
    cost,_,roots=optimal_bst(p,q)

direct=Fraction(1,10)*1+Fraction(2,10)*2+Fraction(5,10)*2+Fraction(1,10)*3+Fraction(1,10)*3
    assert roots[1,2]==1 and cost[1,2]==direct==Fraction(21,10)
    print(f"PASS independent_LCS_pairs={lcs_cases} enumerated_matrix_chains={matrix_cases}
high_q0_root={roots[1,2]} cost={direct}")


def main():
    empty_revenue, empty_choice = rod_cut([0])
    assert empty_revenue == [0] and rebuild_rod(empty_choice, 0) == []
    negative_revenue, negative_choice = rod_cut([0, -2, -5])
    assert negative_revenue == [0, -2, -4]
    assert rebuild_rod(negative_choice, 2) == [1, 1]
    rod_prices = [0, 1, 5, 8, 9]
    revenue, rod_choice = rod_cut(rod_prices)
    pieces = rebuild_rod(rod_choice, 4)
    assert revenue == [0, 1, 5, 8, 10]
    assert rod_choice == [0, 1, 2, 3, 2]
    assert pieces == [2, 2]

    p = [5, 10, 3, 12, 5]
    matrix_cost, split = matrix_chain(p)
    assert matrix_cost[1][4] == 405
    assert split[1][4] == 2
    assert parenthesize(split, 1, 4) == "((A1A2)(A3A4))"

    x, y = "ABCBDAB", "BDCABA"
    lcs_table, directions, lcs_value, lcs_path = lcs_trace(x, y)
    assert lcs_value == "BCBA" and lcs_table[-1][-1] == 4

    p_prob = [Fraction(1, 5), Fraction(3, 10), Fraction(1, 10)]
    q_prob = [
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

```
        Fraction(1, 10),
        Fraction(1, 20),
        Fraction(3, 20),
        Fraction(1, 10),
    ]
    bst_cost, bst_weight, bst_root = optimal_bst(p_prob, q_prob)
    assert bst_cost[1, 3] == Fraction(21, 10)
    assert bst_root[1, 3] == 2

    alignment_score, top, bottom = global_alignment("ACGTA", "AGCAC")
    assert alignment_score[-1][-1] == 1
    assert (top, bottom) == ("ACGTA-", "A-GCAC")

    items = [("A", 3, 5), ("B", 4, 7), ("C", 7, 12)]
    knapsack_values, amount, chosen = unbounded_knapsack(items, 13)
    assert knapsack_values[amount] == 22
    audit_small_dp()
    print(
        "PASS "
        f"rod={revenue} pieces={pieces} "
        f"matrix_cost={matrix_cost[1][4]} split={split[1][4]} "
        f"lcs={lcs_value} lcs_path={lcs_path} "
        f"bst={bst_cost[1, 3]} root={bst_root[1, 3]} "
        f"alignment={alignment_score[-1][-1]} {top}/{bottom} "
        f"knapsack={knapsack_values[amount]} capacity={amount} chosen={chosen}"
    )


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

原創驗算程式附錄: longest_common_subsequence.py

```
"""Recover one LCS and verify it is a common subsequence."""

def lcs_table(x, y):
    table = [[0] * (len(y) + 1) for _ in range(len(x) + 1)]
    for i, xc in enumerate(x, 1):
        for j, yc in enumerate(y, 1):
            if xc == yc:
                table[i][j] = 1 + table[i - 1][j - 1]
            else:
                table[i][j] = max(table[i - 1][j], table[i][j - 1])
    return table


def rebuild(table, x, y):
    out, i, j = [], len(x), len(y)
    while i and j:
        if x[i - 1] == y[j - 1]:
            out.append(x[i - 1])
            i, j = i - 1, j - 1
        elif table[i - 1][j] >= table[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(out))


table = lcs_table("ABCBDAB", "BDCABA")
candidate = rebuild(table, "ABCBDAB", "BDCABA")
assert table[-1][-1] == 4 and len(candidate) == 4
print(candidate)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15 動態規劃 解答 （續）

原創驗算程式附錄: rod_cutting.py

```
"""Bottom-up rod cutting with a reconstruction check."""

def rod_cut(prices):
    n = len(prices) - 1
    revenue = [0] * (n + 1)
    cut = [0] * (n + 1)
    for length in range(1, n + 1):
        candidates = [(prices[first] + revenue[length - first], first) for first in range(1,
length + 1)]
        revenue[length], cut[length] = max(candidates)
    return revenue, cut


prices = [0, 2, 5, 7, 10, 12]
revenue, cut = rod_cut(prices)
assert revenue[5] == 12
length, pieces = 5, []
while length:
    pieces.append(cut[length])
    length -= cut[length]
assert sum(prices[p] for p in pieces) == revenue[5]
print(revenue[5], pieces)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study